```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models

def main():
    # Load and normalize the CIFAR-10 dataset
    print("Loading CIFAR-10 dataset...")
    (train_images, train_labels), (test_images, test_labels) = datasets.cifar10.load_data()
    train_images, test_images = train_images / 255.0, test_images / 255.0

    # Build the CNN model architecture
    print("Building model architecture...")
    model = models.Sequential([
        layers.Input(shape=(32, 32, 3)),

        # Data Augmentation layers
        layers.RandomFlip('horizontal'),
        layers.RandomRotation(0.1),

        # Convolutional Blocks
        layers.Conv2D(32, (3, 3), activation='relu', padding='same'),
        layers.MaxPooling2D((2, 2)),

        layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
        layers.MaxPooling2D((2, 2)),

        layers.Conv2D(64, (3, 3), activation='relu', padding='same'),

        # Classifier Head
        layers.Flatten(),
        layers.Dense(64, activation='relu'),
        layers.Dense(10)  # Output layer with 10 classes (logits)
    ])

    # Compile the model
    model.compile(
        optimizer='adam',
        loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
        metrics=['accuracy']
    )

    model.summary()

    # Train the model
    print("Starting training...")
```

```
    history = model.fit(
        train_images,
        train_labels,
        epochs=10,
        batch_size=64,
        validation_split=0.1
    )

    # Evaluate the model on test data
    print("Evaluating model on test set...")
    test_loss, test_acc = model.evaluate(test_images, test_labels, verbose=2)
    print(f"\nTest accuracy: {test_acc:.4f}")

    # Save the model
    model.save('cifar10_cnn_model.keras')
    print("Model saved successfully.")

if __name__ == '__main__':
    main()
```

2. Training Results Summary

Environment & Dataset Details

- **TensorFlow Version:** 2.21.0

- **Hardware:** CPU execution (GPU Available: False)
- **Dataset:** CIFAR-10 (50,000 training images, 10,000 test images across 10 classes)
- **Model Parameters:** 356,810 total parameters (Trainable: 356,810)

Epoch-by-Epoch Metrics (10 Epochs)

Epoch	Training Loss	Training Accuracy	Validation Loss	Validation Accuracy
1/10	2.3028	0.0979	2.3027	0.0958
2/10	2.3028	0.0986	2.3027	0.0986

3/10	2.3027	0.0993	2.3027	0.0976
4/10	2.3027	0.0980	2.3028	0.0976
5/10	2.3027	0.0969	2.3028	0.0958
6/10	2.3027	0.1003	2.3028	0.0958
7/10	2.3027	0.0999	2.3027	0.0950
8/10	2.3027	0.0998	2.3028	0.0976
9/10	2.3027	0.0964	2.3029	0.0958
10/10	2.3027	0.1000	2.3029	